

PRISM — Review code toàn pipeline

Review nội bộ · commit gốc `7f5bc5d` (2026-09-08) · phạm vi: `prism/src/prism/*` · `prism/scripts/*` · `prism/tests/*` · Docker + repo hygiene · 3.406 dòng Python

Khung pipeline **đúng và chắc**: tách 4 module rõ ràng, gold blocklist chặn leakage ở 3 khoá độc lập, provenance posterior dùng `P_model` thật qua teacher forcing (không one-hot), shrinkage thay cho loại-ô-nhỏ ở cả hai kênh π/v , permutation null riêng từng chuỗi + BH-FDR, và 37 unit test **PASS** (`PYTHONPATH=src python -m unittest discover tests` — 0,08s). Tài liệu `src/prism/README.md` trung thực, kể cả chỗ tự nhận là stub. Doc này **không đề nghị viết lại** gì cả — 24 hạng mục dưới đây là: (1) chỗ số ra sai mà không có lỗi nào báo, (2) chỗ log/comment/config nói khác code, (3) chỗ orchestrator Kaggle tự sửa source lúc chạy. Vị trí ghi theo **tên hàm**, không theo số dòng.

Tóm tắt hạng mục & trạng thái

#	Việc	Repo/Vùng	Ưu tiên	Trạng thái
#1	<code>.gitignore</code> không chặn 2 file zip 1,15 GB đang untracked	repo root	P1	Còn treo
#2	<code>smoke_test</code> ghi đề file kết quả drift thật	src/prism	P1	Còn treo
#3	go/no-go Module C không được code cưỡng chế — bridge NO-GO vẫn dùng	module_c	P1	Còn treo
#4	Vector đặc trưng bridge viết 2 lần ở 2 hàm (lệch thứ tự = w rác)	module_c	P1	Còn treo
#5	<code>p_perm</code> test changepoint nhưng báo cáo là trend (OLS)	module_d	P1	Còn treo
#6	<code>inject_shuffle</code> xáo period theo QUAD, không theo review	eval_injection	P1	Còn treo
#7	Log D-a nói “áp dụng recall_table” trong khi chỉ đọc rồi bỏ	module_d	P1	Còn treo
#8	Ký tự <code> </code> / <code><quad></code> trong term làm mất trắng quad khi parse	module_b_data	P1	Còn treo
#9	<code>find_ckpt</code> chọn <code>selftrain_round1</code> thay vì <code>round2</code>	kaggle_pipeline	P1	Còn treo
#10	Orchestrator và source lúc chạy bằng so khớp chuỗi	kaggle_pipeline	P1	Còn treo
#11	<code>--min-stratum</code> / <code>--min-valence-w</code> : help & config nói “loại ô”, code “shrink”	module_d + config	P2	Còn treo
#12	<code>fit_bridge</code> nhị phân hoá <code>v_image</code> ở 0,5 → mất tín hiệu / crash 1 lớp	module_c	P2	Còn treo
#13	<code>q["score"]</code> or 8.7 biến điểm 0,0 thật thành 8,7	module_c	P2	Còn treo
#14	Tiêu chí dừng self-train không so với teacher gốc	module_b_selftrain	P2	Còn treo
#15	Pseudo-label bỏ hẳn unit rỗng → student học over-generate	module_b_selftrain	P2	Còn treo
#16	<code>--limit</code> lấy N unit ĐẦU FILE, mọi vòng self-train dùng cùng slice	module_b_infer	P2	Còn treo
#17	<code>by_language</code> : bucket “other” luôn rỗng, review khác en/vi bị bỏ đếm	module_b_eval	P2	Còn treo
#18	Probe nhân bản WEST/CODES và đã lệch khỏi <code>config</code>	scripts/probe	P2	Còn treo
#19	<code>COHORT_DEFS</code> là code chết; ngưỡng cohort hardcode trong Module A	config	P2	Còn treo
#20	<code>find_one</code> / <code>clone_repo</code> chọn file & repo tùy ý theo glob	kaggle_pipeline	P2	Còn treo
#21	Docker không pin version; tên file <code>vimg</code> lệch giữa README và orchestrator	Docker + README	P2	Còn treo
#22	Ảnh tải về không kiểm tra nội dung → JPEG hỏng đếm là “đã có”	download_pool_photos	P2	Còn treo
#23	<code>load_quads</code> tra <code>CODE2CAT</code> trước khi bỏ quad <code>w ≤ 0</code> → KeyError	module_d	P3	Còn treo
#24	Dọn nhỏ: chrono split, magic number, hàm trùng, assert số cứng	toàn repo	P3	Còn treo

Phần A — Đường găng ra số của paper

Những chỗ pipeline vẫn chạy xanh nhưng con số cuối cùng sai hoặc không đúng như mô tả.

`src/prism/smoke_test.py`

#2 — Smoke test ghi đè file kết quả drift thật P1

VỊ TRÍ `main` (bước 4 và bước 5)

VÌ SAO Bước 4 gọi `module_d_drift` **không truyền `--out`**, nên nó ghi đúng đường dẫn mặc định `outputs/drift/drift_results.corpus.taxonomy_code.json` — chính là file kết quả chính thức của cohort corpus. Bước 5 lại ghi tiếp `injection_shuffle.json` + `drift_results.inj_shuffle_r*.json`. Kịch bản: chạy corpus thật xong (nhiều giờ), rồi chạy `python -m prism.smoke_test` để kiểm tra sau khi sửa code → file kết quả bị thay bằng 50k pseudo-quad keyword tổng hợp, **cùng tên, cùng schema, không cảnh báo**. Không có backup vì `outputs/` nằm trong `.gitignore`.

SỬA Truyền `--out` trở vào `outputs/drift/smoke/drift_results.smoke.json`, và set `PRISM_WORK_DIR` tạm (hoặc thêm `--out-dir` cho `eval_injection`) để mọi sản phẩm smoke nằm trong thư mục riêng.

`src/prism/module_c_reliability.py`

#3 — Ngưỡng go/no-go được tính rồi bỏ, không chặn gì P1

VỊ TRÍ `fit_bridge` → `apply_weights`

VÌ SAO `fit_bridge` tính Spearman với human audit, ghi `go_nogo: "NO-GO"` vào `bridge_report.json` — rồi **vẫn pickle `bridge.pkl` ra đĩa**. `apply_weights` quyết định dùng bridge chỉ bằng `if bp.exists()`. Hệ quả: $p = 0,05$ (dưới ngưỡng 0,30) vẫn cho ra `w = f(bridge)` cho toàn corpus, và hợp đồng ghi trong README ("NO-GO → fallback `conf_seq`") **không bao giờ chạy**. Tệ hơn: nếu `len(pairs) < 30` (audit chưa gán đủ, hoặc khoá không khớp — xem Phần C) thì khối tính Spearman bị bỏ qua im lặng, `go_nogo` không có trong report, và bridge vẫn được dùng.

SỬA Ghi verdict vào **trong** pickle; `apply_weights` đọc verdict và fallback `conf_seq` nếu $\neq$ GO. Khi NO-GO hoặc thiếu audit thì đặt tên file `bridge_NOGO.pkl` để không có đường dùng nhầm. Tương tự cho `verifier.pkl`: `verifier_report.json` có Δ AUC nhưng `apply_verifier` không kiểm tra.

#4 — Vector đặc trưng bridge được viết tay 2 lần P1

VỊ TRÍ `fit_bridge.feats` (hàm nội bộ) vs list literal trong `apply_weights.rows`

VÌ SAO Hai nơi dựng cùng vector 5 chiều [`conf_seq`, `p_posterior`, `phi_pos`, `loglp(n_words)`, `prov_flip`] bằng hai đoạn code độc lập. Hiện tại thứ tự trùng nhau nên đúng. Thêm 1 đặc trưng (ví dụ `has_photo`) vào `feats` mà quên sửa `apply_weights` → sklearn nhận đủ 5 cột, **không raise**, chỉ dịch sai cột: `f` rác → `w` rác → mọi số π/v , mọi FDR, mọi verdict trong paper sai mà pipeline chạy xanh hoàn toàn. Đây là loại lỗi không test nào hiện có bắt được.

SỬA Nâng `feats` thành hàm module-level `bridge_features(q) → list[float]` đặt cạnh `BRIDGE_FEATURE_NAMES`, gọi từ cả hai chỗ; thêm assert `len(x) == len(BRIDGE_FEATURE_NAMES)` và so `bridge.n_features_in_` khi load pickle.

#12 — Nhị phân hoá `v_image` ở 0,5 làm mất tín hiệu và có thể crash P2

VỊ TRÍ `fit_bridge` (dòng dựng `yb`)

VÌ SAO `v_image` là xác suất liên tục do `head.predict_proba` sinh ra, bị ép về 0/1 tại 0,5. (a) Mất toàn bộ độ tin cậy — quad `v_image=0,51` và `0,99` thành cùng một nhãn. (b) Head được fit trên tập cân bằng 1:1 nhưng pool **không** cân bằng; nếu mọi `v_image` rơi cùng một phía 0,5 thì `LogisticRegression.fit` raise "needs samples of at least 2 classes" và stage bridge chết sau khi đã embed xong toàn bộ ảnh. Cùng rủi ro với model propensity nếu `has_photo` đồng nhất trong file đầu vào.

SỬA Fit trực tiếp trên nhãn mềm (hồi quy logistic có `sample_weight` nhân đôi hàng theo `v_image`, hoặc beta-regression), tối thiểu là guard `len(set(yb)) < 2` với thông báo nói rõ phân phối `v_image` quan sát được.

#13 — `q["score"]` or 8.7 biến điểm 0,0 thật thành 8,7 P2

VỊ TRÍ `fit_bridge` (đặc trưng propensity `Xp` và `w_ipw`)

VÌ SAO `U.parse_score("Đạt điểm 0,0")` trả đúng `0.0` (đã kiểm chứng), và `0.0 or 8.7 == 8.7` trong Python. Nên review bị chấm 0 điểm — **đúng cái đuôi quan trọng nhất cho drift phân nản** — được gán điểm 8,7 (mean của review có ảnh) trong cả model propensity lẫn khi áp IPW. Kết quả: trọng số IPW của nhóm điểm thấp bị lệch có hệ thống, không có cảnh báo.

SỬA `q["score"] if q["score"] is not None else 8.7`, tách hằng số ra `config` (đang hardcoded 2 lần trong cùng hàm).

#12b — `w = conf_seq` đã temperature-scale chỉ có trong docstring P3

VỊ TRÍ docstring module `module_c_reliability` vs `apply_weights`

VÌ SAO Header module và README §4 nói fallback NO-GO là "`conf_seq` đã temperature-scale"; `apply_weights` dùng `r_hat = q["conf_seq"]` thô, không có bước hiệu chỉnh nhiệt độ nào trong repo. Người đọc paper sẽ tưởng nhánh fallback đã được calibrate.

SỬA Hoặc implement (fit 1 tham số T trên dev), hoặc sửa docstring + README thành "`conf_seq` thô, chưa calibrate".

`src/prism/module_d_drift.py`

#5 — `p_perm` kiểm định changepoint, nhưng được báo cáo cùng slope OLS như một kết luận trend P1

VỊ TRÍ `channel_stats` → `permutation_p` → khối verdict trong `run`

VÌ SAO `channel_stats` tính hai thống kê khác nhau: `slope_per_year / t_stat` từ `ols_trend`, và `welch_t` từ `best_split`. `permutation_p` chỉ nhận `obs_t = ct` (Welch của changepoint) — nên `p_perm`, và do đó `p_fdr` + `significant_after_fdr` + `n_significant_after_fdr`, **hoàn toàn là kiểm định bước nhảy**. Nhưng verdict lại gate trên dấu slope OLS (`sa * sr > 0`). Hai hệ quả cụ thể: một aspect trôi đơn điệu mạnh (OLS $t = 8$) nhưng không có bước nhảy rõ → `p_perm` cao → verdict “phẳng”; ngược lại một bước nhảy sạch với slope ≈ 0 → “XU HƯỚNG THẬT”. Con số headline “*k/14 aspect vượt FDR*” là số aspect có changepoint, đang được gọi là số aspect có xu hướng.

SỬA Chạy permutation cho **cả hai** thống kê (cùng vòng lặp, thêm `ols_trend(deseason(sh))` vào `permutation_p`), trả `p_perm_trend` và `p_perm_cp` riêng, đưa vào 2 lưới FDR riêng; verdict gate trên `p_perm_trend`. Nếu quyết định giữ nguyên thì phải đổi tên trường thành `p_perm_changepoint` và sửa README + caption bảng trong paper.

#7 — Log nói “áp dụng recall_table.json”, code chỉ đọc rồi bỏ P1

VỊ TRÍ `run` (khối D-a)

VÌ SAO `rec = json.loads(rec_path.read_text())` gán vào biến rồi **không dùng ở đâu nữa** (đã grep toàn repo: `rec` không xuất hiện lần thứ hai). Log in ra “D-a: áp dụng recall_table.json”. README §5 có ghi rõ đây là stub — nên vấn đề nằm ở log: khi tập audit D0 về và ai đó đặt file recall vào `outputs/drift/`, log sẽ báo đã áp dụng, số ra không có hiệu chỉnh recall, và không cách nào phát hiện từ output.

SỬA Đổi log thành “D-a: THẤY recall_table.json nhưng CHƯA áp dụng (stub) — số chưa hiệu chỉnh recall” ở mức `WARNING`, ghi cờ “`recall_adjusted`”: `false` vào JSON output, và `raise SystemExit` nếu chạy với `--final`.

#11 — `--min-stratum / --min-valence-w`: help và config nói “loại ô”, code “shrink” P2

VỊ TRÍ `main` (help của argparse), `config.MIN_STRATUM_N` / `config.MIN_VALENCE_CELL_W`

VÌ SAO Code đã đổi sang shrinkage (đúng, và có test pin lại — xem Phần C), nhưng comment trong `config.py` vẫn viết “ô dưới ngưỡng này bị bỏ qua trong kỳ đó” và help của `--min-valence-w` vẫn viết “ô ... dưới tổng trọng số này bị bỏ ở kênh v”. Hệ quả cụ thể: ai đó muốn siết lọc sẽ đặt `--min-stratum 200` và nhận điều ngược lại — prior mạnh gấp 7 lần, mọi share bị kéo về share gộp của kỳ, **mọi trend bị làm phẳng**, và họ sẽ đọc kết quả đó như “không có drift”. README §5 mô tả đúng; chỉ config + help sai.

SỬA Đổi 2 comment trong config và 1 help thành “cường độ prior shrink về share/tỷ lệ gộp của kỳ (đơn vị: tổng w)”. Cần nhắc đổi tên biến thành `STRATUM_PRIOR_W` / `VALENCE_PRIOR_W` để tên không còn gợi ý ngưỡng lọc.

#23 — `CODE2CAT` được tra trước khi quad `w ≤ 0` bị bỏ P3

VỊ TRÍ `load_quads`

VÌ SAO Thứ tự trong vòng lặp: tính `a` (có `C.CODE2CAT[q["taxonomy_code"]]`) **trước** `if w ≤ 0: continue`. `apply_weights` chú ý đặt `w=0` cho code ngoài taxonomy, nhưng Module D crash `KeyError` trước khi tới nhánh bỏ đó. Hiện an toàn *chỉ vì* `parse_linearized` đã lọc code ở bước parse — bất kỳ file quad dựng tay (injection, probe, mock test) có code lạ sẽ làm chết cả run ở một dòng bất kỳ giữa 1,9M dòng.

SỬA Chuyển `w` và `if w ≤ 0: continue` lên trước, và dùng `C.CODE2CAT.get(code)` + `continue` có đếm thống kê thay vì index trực tiếp.

#23b — “ĐẢO DẤU sau hiệu chỉnh” cũng nổ khi slope bằng 0 P3

VỊ TRÍ `run` (khối verdict)

VÌ SAO `sr = (...).get("slope_per_year") or 0.0` gộp “thiếu số” và “bằng 0” thành một; nhánh `elif a_sig` cuối cùng nhận cả trường hợp `sa * sr == 0` và gán nhãn “ĐẢO DẤU”, vốn là một kết luận rất mạnh trong paper. Cũng vậy: `ols_trend` chia cho `sxx` không guard — chuỗi 1 điểm (đường bootstrap nếu `pi_periods` co lại) chia cho 0.

SỬA Dùng `is None` tường minh; thêm nhánh riêng “không xác định” khi một trong hai slope thiếu; guard `sxx == 0`.

`src/prism/eval_injection.py`

#6 — `inject_shuffle` xáo period theo QUAD, phá cấu trúc review P1

VỊ TRÍ `inject_shuffle` (dùng bởi E4, và bởi E3a để dựng nền null)

VÌ SAO Hàm shuffle **list period** rồi **zip** lại theo từng quad. Hai quad của cùng một **review_uid** vì thế nhận hai tháng khác nhau — dữ liệu null chứa những review không tồn tại, trải trên nhiều kỳ. Hai hệ quả: (a) `load_quads` gom `rev_acc[(period, stratum)][review_uid]`, nên bootstrap resample “review” trên các mảnh review bị xé, tức đơn vị resample nhỏ hơn thật; (b) với E4, xáo theo quad **phá tương quan trong review**, làm null độc lập hơn dữ liệu thật → phương sai chuỗi bị hạ → **FPR thực nghiệm bị ước lượng thấp** và negative control PASS quá dễ. Đúng chỗ này là điểm tựa cho toàn bộ lập luận “khung không sinh drift giả”, nên sai lệch ở đây là sai lệch của kết luận chính.

SỬA Xáo ở mức review: lấy `uids = sorted({q["review_uid"]})`, dựng `period_of_uid`, permute **giá trị** của map đó, rồi broadcast xuống mọi quad của cùng uid. Test `test_preserves_period_multiset_and_content` vẫn pass, cần thêm 1 test: mọi quad cùng uid có cùng period sau khi xáo.

#6b — `inject_valence` tự nhận “đổi đúng một thứ” nhưng đổi cả kênh π P2

VỊ TRÍ `inject_valence` + header module (E3c)

VÌ SAO Header viết “*lật 5% quad pos→neg ... KHÔNG đổi φ — injection chỉ đổi đúng một thứ*”. Nhưng `load_quads` nhận quad vào kênh π theo điều kiện `phi == "NEG" or sentiment == "negative"`, nên mọi quad bị lật cũng **đi vào tử số của π** . Cột `prevalence_detected_within_3` trong report E3c vì thế bị nhiễu bởi thiết kế: nó không phải bằng chứng “ π giữ im lặng khi chỉ có valence drift”, mà là hệ quả tất yếu của injection.

SỬA Nói rõ trong report (thêm trường “*note*”: “ π cũng bị ảnh hưởng theo thiết kế”), hoặc thêm `--pi-uses-phi-only` cho Module D để E3c đo được kênh π độc lập với sentiment.

`src/prism/module_b_data.py · module_b_selftrain.py · module_b_infer.py · module_b_eval.py`

#8 — Ký tự | hoặc <quad> trong term làm mất trắng cả quad P1

VỊ TRÍ `linearize` ↔ `parse_linearized`

VÌ SAO Đã kiểm chứng: `linearize([{"aspect_term": "gia | chat luong", ...}])` sinh `<quad> gia | chat luong | FAC_ROOM | tot | positive </quad>`, và `parse_linearized` của chuỗi đó trả `[]` — 5 field ≠ 4 nên quad bị bỏ im lặng. Tương tự nếu term chứa `</quad>`. Hai đường thiết hại: (a) lúc train, target vẫn chứa quad đó nên model được dạy sinh ra format mà chính parser sẽ bỏ; (b) lúc infer, mọi quad model sinh có `|` trong opinion (rất thường trong review đa ngữ: “giá | chất lượng”, “wifi|tv”) bị mất → recall lệch có hệ thống theo *loại term*, không theo loại aspect, nên không xuất hiện trong bảng `by_category` nào. `roundtrip_mismatch` có đếm và smoke test có assert `== 0` — hiện gold sạch nên bằng 0; nhưng đó là bằng chứng cho gold hôm nay, không phải bảo vệ cho pool.

SỬA Escape trong `linearize` (thay `|` → `|`; hoặc bỏ hẳn ký tự phân cách khỏi term) và unescape/khôi phục trong `parse_linearized`; đồng thời `split("|", 3)` với `maxsplit` để field cuối (sentiment) luôn parse được. Thêm test round-trip cho term có `|` và `</quad>`.

#14 — Self-train không so dev F1 với teacher gốc P2

VỊ TRÍ `module_b_selftrain.main` (khối bước 4)

VÌ SAO Điều kiện dừng là `len(history) > 1 and f1 < history[-2]["dev_f1"]`: vòng 1 **không có** mốc nào để so, nên luôn được nhận và `ckpt = new_ckpt`. Nếu pseudo-label vòng 1 làm dev F1 tụt từ 0,62 xuống 0,55, script vẫn báo thành công, ghi `final_ckpt = selftrain_round1`, và `infer` chính thức dùng model **tệ hơn seed**. Không log nào nói ra vì seed chưa từng được eval trong cùng script.

SỬA Eval seed teacher **trước** vòng 1, đưa vào `history[0]` với `round: 0`, và chỉ nhận `new_ckpt` khi `f1 > max(dev_f1 đã thấy)`.

#15 — Pseudo-label bỏ hẳn unit không còn quad nào P2

VỊ TRÍ `pseudo_rows`

VÌ SAO `by_unit` chỉ nhận quad vượt `tau / tau_post`, và `rows` được dựng từ `by_unit.items()` — nên một (review, φ) mà model sinh 3 quad đều dưới ngưỡng **biến mất hoàn toàn** khỏi tập train, thay vì trở thành ví dụ target rỗng. Student vì thế chỉ thấy input “luôn có quad để trích”, học over-generate: precision trên gold test tụt trong khi recall tăng — đúng dấu hiệu error propagation mà guardrail `--max-skew` không phát hiện được (nó chỉ xem phân phối cực tính, không xem tỷ lệ unit rỗng).

SỬA Giữ unit rỗng với `target = ""` theo một tỷ lệ định trước (ví dụ bằng tỷ lệ segment rỗng trong gold), và log `n_empty_units` vào `selftrain_history.json`.

#16 — `--limit` lấy N unit đầu file, mọi vòng self-train dùng cùng slice P2

VỊ TRÍ `module_b_infer.main` (vòng lặp `units()`) ← gọi từ `module_b_selftrain.main`

VÌ SAO `units()` đọc `reviews.jsonl.gz` theo thứ tự ghi của Module A (theo thứ tự pool, tức gom theo hotel), và `--limit` cắt N đơn vị **đầu tiên**. Hai hệ quả: (a) 200k đơn vị đó **không phải mẫu ngẫu nhiên** của cohort — chúng thuộc một tập con hotel xuất hiện sớm trong pool, nên phân phối stratum/kỳ bị lệch, và pseudo-label thừa hưởng đúng cái lệch đó; (b) vòng 2 của self-train gọi lại infer với cùng `--limit` nên **đọc lại đúng cùng 200k đơn vị** — không có dữ liệu mới, chỉ có teacher mới, đúng công thức để tự khuếch đại lỗi của chính mình.

SỬA Thêm `--sample-seed` + reservoir sampling trên `units()` (đã có sẵn pattern Algorithm R trong `make_audit_samples`), và cho self-train truyền seed khác nhau mỗi vòng (`C.RANDOM_SEED + rnd` — đã có biến này).

#17 — `by_language`: bucket “other” luôn rỗng P2

VỊ TRÍ `module_b_eval.main` (dict `report["by_language"]`)

VÌ SAO Bucket được duyệt trên 3 chuỗi literal (`"en", "vi", "other"`) và filter so `langs.get(...) == lv`, trong khi `langs` chứa mã ngôn ngữ thật (`"ko", "zh", "fr", "?"` ...). Không review nào có `languages[0] == "other"`, nên cột “other” luôn `gold=0 pred=0 F1=0`, và **toàn bộ review không en/vi bị bỏ khỏi bảng** — tức 3,3% gold (theo tỷ lệ 80,5 en / 16,2 vi trong docstring train) không được báo cáo, và F1 theo ngôn ngữ không cộng lại thành overall.

SỬA `"other": agg(lambda g: langs.get(g["source_review_id"]) not in ("en", "vi"))`, và thêm `assert` mềm rằng `gold(en)+gold(vi)+gold(other) == gold(overall)`.

Phần B — Orchestration, cấu hình, repo

Không ảnh hưởng công thức, nhưng đây là chỗ một run 6 tiếng GPU chết giữa đường hoặc chạy sai checkpoint.

repo root · `.gitignore`

#1 — Hai file zip 1,15 GB đang untracked và KHÔNG bị gitignore P1

VỊ TRÍ `prism/.gitignore` (repo không có `.gitignore` ở root)

VÌ SAO `git status` đang liệt `?? hamos-mabsa.zip` (971 MB) và `?? prism/data/raw.zip` (207 MB). `.gitignore` chỉ chặn `/data/raw/*.jsonl` — không chặn `.zip`, và không chặn gì ở root vì file `.gitignore` nằm trong `prism/`. Một lệnh `git add -A` (rất dễ xảy ra khi commit message toàn là “update”) sẽ đẩy 1,15 GB vào history — không xóa được bằng `git rm`, phải rewrite history, và GitHub chặn file >100 MB nên push sẽ fail *sau khi* commit đã hình thành.

SỬA Thêm `.gitignore` ở root repo với `*.zip + hamos-mabsa/`, và thêm `*.zip` vào `prism/.gitignore`. Làm việc này **trước** mọi commit khác.

`scripts/kaggle_pipeline.py`

#9 — `find_ckpt` chọn `selftrain_round1` thay vì `round2` P1

VỊ TRÍ `find_ckpt.rank`

VÌ SAO Đã kiểm chứng: với hai thư mục `.../selftrain_round1` và `.../selftrain_round2`, khoá rank là `(..., len(d))` — độ dài bằng nhau — nên `sorted(set(dirs))` quyết định theo thứ tự chữ và **round1 thắng**. README ghi bất biến “*infer chính thức phải dùng checkpoint tốt nhất từ self-train*”; orchestrator làm ngược lại, in ra `CKPT = ...round1` giữa hàng trăm dòng log, rồi chạy 6+ tiếng infer trên corpus bằng model của vòng trước.

SỬA Đọc `selftrain_history.json` (đã có `final_ckpt + dev_f1` từng vòng) và ưu tiên `final_ckpt`; fallback: rank theo số vòng parse từ tên (`round(d+)`) **giảm dần**, rồi theo `mtime`. In dev F1 của checkpoint được chọn.

#10 — Orchestrator ghi đè source code lúc chạy bằng so khớp chuỗi P1

VỊ TRÍ `patch_selftrain_batch` và `patch_infer_bug` (gọi từ `main / do_step`)

VÌ SAO `patch_selftrain_batch` tìm một needle 2 dòng **kèm đúng 24 khoảng trắng thật lẽ** trong `module_b_selftrain.py` và `write_text` đè lại file để chèn `--batch / --score-batch`. Needle hiện còn khớp (đã kiểm chứng). Reformat, đổi tên biến, hay chỉ cần `black` chạy qua file đó → needle không khớp → `if needle in t false` → **không patch, không cảnh báo** → self-train gọi infer với default `--batch 32 --score-batch 48` → OOM trên T4 sau khi đã train xong vòng 1, mất cả session GPU. `patch_infer_bug` thì đã là no-op hoàn toàn: needle `s_post != q["sentiment_model"]` không còn tồn tại trong `module_b_infer.py` (đã kiểm chứng: `False`) — nó chỉ còn là code chết có quyền ghi vào source.

SỬA Thêm `--batch` và `--score-batch` làm argparse thật của `module_b_selftrain`, forward xuống `subprocess.run`, rồi **xoá cả hai hàm patch**. Không có lý do nào để orchestrator được quyền viết vào `src/`.

#20 — `find_one` và `clone_repo` chọn file/repo tùy theo thứ tự glob P2

VỊ TRÍ	<code>find_one (return hits[0])</code> và <code>clone_repo</code> (vòng lặp <code>SEARCH_ROOTS</code>)
VÌ SAO	Trên Kaggle rất thường xuyên attach cùng lúc dataset store + output notebook bước trước, và cả hai đều chứa <code>train.t2t.jsonl / dev.jsonl / hotel_cohorts.json</code> . <code>glob(recursive=True)</code> không bảo đảm thứ tự, nên <code>hits[0]</code> có thể là bản cũ; step <code>train</code> sẽ stage file cũ vào <code>work/extract/</code> , in <code>staged ...</code> như bình thường, và train trên split sai. Cùng vấn đề với <code>clone_repo</code> : nó ưu tiên bất kỳ repo tìm thấy dưới <code>/kaggle/input</code> trước khi clone, nên một output notebook cũ có thư mục <code>Prism</code> sẽ khiến cả run dùng code cũ — trong khi cell setup ở README vừa <code>git clone --depth 1</code> bản mới về <code>/kaggle/working</code> .
SỬA	<code>find_one</code> : sắp theo (ưu tiên <code>/kaggle/working</code> trước <code>/kaggle/input</code> , rồi <code>mtime</code> giảm dần), và in ra tất cả hit khi có >1 để người chạy thấy mình đang lấy bản nào. <code>clone_repo</code> : đảo thứ tự — thử clone trước, chỉ fallback sang repo có sẵn khi không có internet, và in commit hash đang dùng.

`src/prism/config.py`

#19 — `COHORT_DEFS` là code chết, ngưỡng cohort thật hardcoded trong Module A P2

VỊ TRÍ	<code>config.COHORT_DEFS</code> vs <code>module_a_store.build_store</code> (khối A8)
VÌ SAO	Đã grep toàn repo: <code>COHORT_DEFS</code> và <code>needs_gold</code> chỉ xuất hiện trong chính <code>config.py</code> , không nơi nào đọc. Ngưỡng thật nằm trong <code>build_store</code> dưới dạng literal <code>≥ 1000 / ≥ 300 / sp == "test"</code> . Docstring đầu file config viết “Mọi hằng số/ngưỡng của pipeline nằm ở đây, không rải rác trong code” — câu đó sai đúng ở cái knob dễ bị tinh chỉnh nhất. Ai sửa <code>min_pool</code> trong <code>COHORT_DEFS</code> sẽ thấy cohort size không đổi và đi tìm bug ở chỗ khác. Ngoài ra “T-unbiased” thiếu khóa <code>needs_gold</code> mà 3 entry kia đều có → consumer đầu tiên viết <code>defs["needs_gold"]</code> sẽ ăn <code>KeyError</code> .
SỬA	Cho <code>build_store</code> đọc <code>C.COHORT_DEFS</code> (một vòng lặp thay 4 dòng literal), thêm <code>needs_gold=True</code> cho <code>T-unbiased</code> . Hoặc nếu quyết định giữ hardcoded thì xoá <code>COHORT_DEFS</code> hẳn — đừng để hai nguồn sự thật.

`scripts/probe_complaint_composition.py`

#18 — Probe nhân bản hằng số và đã lệch khỏi `config` P2

VỊ TRÍ	<code>WEST / ASIA / CODES / MIN_STRATUM / bloc</code> ở đầu file
VÌ SAO	Probe khai báo lại toàn bộ taxonomy strata thay vì <code>from prism import config</code> , và đã lệch (đã kiểm chứng): <code>config.WEST_COUNTRIES</code> có “phần lan”, probe không → mọi review Phần Lan vào bloc <code>OTH</code> ở probe và <code>WEST</code> ở pipeline. <code>CODES</code> của probe có thêm <code>AM_WIFI</code> , <code>FAC_CLIMATE</code> , <code>AM_TRANSPORT</code> và thiếu <code>EXP_OVERALL</code> , <code>LOY_RETURN</code> , <code>FAC_INTERIOR</code> , <code>EXP_VALUE</code> so với <code>CODES_REPORTABLE</code> . Hệ quả: con số khả thi trích trong <code>docs/approach-temporal-trend.md</code> (“8/13 aspect đối kết luận sau hiệu chỉnh”, mùa vụ 2,22× / 1,73× / 1,05×) được tính trên định nghĩa strata và tập aspect khác với Module D — nên không tái lập được bằng cách chạy pipeline, và nếu reviewer paper yêu cầu đối chiếu thì hai bảng sẽ không khớp mà không ai giải thích được vì sao.
SỬA	Import từ <code>prism.config + prism.utils (country_bloc , length_bin , make_stratum</code> đã có sẵn và đã có test), xoá bản sao. Nếu probe có ý dùng 13 code khác thì khai báo thành <code>PROBE_CODES</code> kèm 1 dòng lý do và ghi rõ trong docs.

`scripts/download_pool_photos.py` · Docker

#22 — Ảnh tải về không kiểm tra nội dung; JPEG hỏng được đếm là “đã có” P2

VỊ TRÍ	<code>main</code> (khối <code>urlopen</code> → <code>dest.write_bytes</code>)
VÌ SAO	<code>resp.read()</code> được ghi thẳng ra <code>.jpg</code> mà không xem <code>Content-Type</code> hay magic bytes. CDN trả trang HTML lỗi 200 (rất thường với ảnh Booking đã hết hạn) → file HTML mang tên <code>.jpg</code> , kích thước > 0. Lần chạy lại, nhánh resume <code>dest.exists() and size > 0</code> tính nó là <code>n_skip</code> (“đã có”) và đưa vào index. Sang <code>module_c.apply_verifier</code> , <code>Image.open</code> ném exception, bị bắt bởi <code>except Exception: feats[uid] = None</code> → quad đó không có <code>v_image</code> , không có counter nào ghi lại . Kết quả: log “v_image cho N/M quad” thấp bất thường và không có cách nào biết vì URL chết, vì ảnh hỏng, hay vì cohort nhỏ.
SỬA	Kiểm <code>resp.headers.get("Content-Type").startswith("image/")</code> + magic bytes JPEG/PNG trước khi ghi; ghi ra file tạm rồi <code>rename</code> (ghi nửa với hiện tại vẫn để lại file); trong <code>apply_verifier</code> đếm riêng <code>n_decode_fail</code> và log ra. Phụ: <code>if args.limit and n_new ≥ args.limit: continue</code> nên <code>break</code> khi không cần quét tiếp để index (hiện quét hết 1,9M dòng sau khi đã đủ limit).

#21 — Docker không pin version; tên file vimg lệch giữa README và orchestrator P2

VỊ TRÍ	<code>Dockerfile.torch</code> (lệnh <code>pip install</code>) · <code>kaggle_pipeline.do_step</code> (nhánh <code>c_apply_verifier</code> / <code>c_bridge</code>) vs README §4
VÌ SAO	(a) <code>pip install torch transformers sentencepiece accelerate</code> không pin version nào, và <code>requirements.txt</code> chỉ có <code>≥</code> . Với một artifact kèm paper, build lại image sau 6 tháng cho ra transformers khác major → <code>generate()</code> / <code>num_beams</code> đổi hành vi và số không tái lập. (b) README §4 ghi output apply_verifier là <code>outputs/extract/pool_quads.T-unbiased.vimg.jsonl.gz</code> , còn orchestrator dùng <code>outputs/reliability/pool_quads_vimg.T-unbiased.jsonl.gz</code> — và step <code>c_bridge</code> tìm file theo đúng tên qua <code>copy_in(vimg, ...)</code> . Ai chạy local theo README rồi upload output lên Kaggle sẽ nhận <code>FileNotFoundError: Không thấy pool_quads_vimg.T-unbiased.jsonl.gz</code> dù file có đó, tên khác. (c) <code>.dockerignore</code> loại <code>tests/</code> + <code>scripts/</code> , nên image sinh ra kết quả không chạy được unit test lẫn smoke test.
SỬA	Pin <code>==</code> cho torch/transformers/open_clip trong <code>requirements.txt</code> và dùng <code>pip install -r</code> trong <code>Dockerfile.torch</code> ; thống nhất một tên duy nhất cho file vimg (đặt hằng số trong <code>config.py</code> và dùng ở cả README, module C, orchestrator); bỏ <code>tests/</code> khỏi <code>.dockerignore</code> .

Dọn nhỏ — gộp thành 1 commit

#24 — Bốn hạng mục P3 độc lập P3

VỊ TRÍ	<code>module_b_data.build</code> · <code>module_b_selftrain.main</code> · <code>module_b_infer.seq_logprobs</code> / <code>seq_conf</code> · <code>smoke_test.main</code> · <code>module_c.train_verifier</code>
VÌ SAO	(a) chrono split : <code>r["review_date"][:7] ≤ "2024-06"</code> — review thiếu ngày có <code>review_date == ""</code> , và <code>""[:7] == ""</code> nhỏ hơn mọi chuỗi, nên mọi review thiếu ngày rơi vào <code>chrono_train</code> (và bị loại khỏi <code>chrono_test</code>). Probe E1c vì thế có tập early bị nhiễm dữ liệu không xác định thời gian. (b) magic number : <code>gold_neg_rate = 0.154</code> hardcoded trong self-train — nếu gold được cập nhật, guardrail <code>--max-skew</code> so với một con số đã chết; tính từ <code>gold_train</code> ngay tại chỗ. (c) hàm trùng : <code>seq_logprobs</code> và <code>seq_conf</code> giống nhau từng dòng trừ 2 dòng cuối — hai bản sao đồng nghĩa hai lần phải sửa khi đổi <code>max_length</code> (hiện <code>160 / 192</code> hardcoded ở 4 chỗ trong file, độc lập với <code>--max-src</code> / <code>--max-tgt</code> của train). (d) assert số cứng : <code>assert len(hs) == 3399</code> và <code>== 23995</code> trong smoke test — cập nhật gold hợp lệ làm smoke fail với <code>AssertionError</code> không thông điệp, dễ bị hiểu là code hỏng. (e) output chết : <code>train_verifier</code> ghi <code>image_feats.npy</code> + index mà không ai đọc, và <code>np.stack([])</code> raise nếu không ảnh nào resolve (thay vì báo "không thấy ảnh gold ở <code>HAMOS-ROOT</code> "); <code>build_verifier_dataset</code> chia cho <code>len(rows)</code> → <code>ZeroDivisionError</code> cùng tình huống.
SỬA	(a) lọc <code>if not r["review_date"]</code> ra một nhánh <code>chrono_unknown</code> và log số lượng; (b) tính từ dữ liệu; (c) gộp thành <code>_seq_token_logprobs()</code> trả tensor, hai wrapper mỏng, đưa <code>160/192</code> thành hằng số module; (d) đọc kỳ vọng từ <code>tests/expected_counts.json</code> với message chỉ vào file đó; (e) xoá output không dùng, guard <code>if not feats: raise SystemExit(...)</code> .

Phần C — Đừng đụng vào

Đang đúng, và dễ bị phá đúng lúc refactor. Mỗi mục: nó là gì + cái gì phá nó.

U.quad_uid — khoá nối duy nhất giữa mẫu audit người và bridge

`make_audit_samples.quad_audit_sample` ghi `quad_uid` vào file cho người gán, và `module_c.fit_bridge` **đựng lại** khoá đó từ pool quad bằng cùng hàm để join. Đổi format (thêm field, đổi `[:6]` của hash, đổi `norm_text`) sau khi audit đã gán xong → `pairs` rỗng → `len(pairs) ≥ 30` false → khối Spearman **bị bỏ qua im lặng**, `go_nogo` không xuất hiện trong report, và bridge vẫn được ghi + dùng (xem #3). Không có test nào bảo vệ parity này — hãy thêm một test dùng chung một dict quad qua cả hai đường.

Cờ in_gold và 3 điểm chặn độc lập của nó

`module_a_store.is_gold_leak` chặn bằng 3 khoá (hotel×date, hash 120 ký tự đầu, map hotel→split) và chỉ **gắn cờ**, không xoá dòng. Việc loại thật xảy ra ở **ba chỗ riêng biệt**: `module_b_infer.units`, `download_pool_photos.main`, `make_audit_samples.do_sample`. Bỏ bất kỳ một trong ba `if r["in_gold"]` nào là mở đường gold vào pseudo-label/train — và pipeline vẫn chạy xanh, F1 chỉ đẹp lên. Nếu thêm consumer mới của `reviews.jsonl.gz`, chặn ngay ở dòng đầu vòng lặp.

U.benjamini-hochberg — step-up có `prev = min(...)`

Duyệt **ngược** từ p lớn nhất và giữ running-min để bảo đảm q-value đơn điệu. "Đơn giản hoá" thành `p * n / rank` không có running-min sẽ cho q không đơn điệu (một p nhỏ hơn có thể nhận q lớn hơn) và số aspect vượt FDR đổi. `test_bh_known_example` pin lại đúng tính chất này.

Shrinkage ô strata nhỏ (không phải loại ô) ở cả hai kênh

`shares_by_period` và `valence_by_period` shrink ô nhỏ về share/tỷ lệ gộp của kỳ (prior Dirichlet/Beta cường độ `min_n / min_w`). Quay về loại-ô nhỏ làm mix strata đóng góp **tự trôi theo thời gian** khi một stratum teo dần → sinh trend giả *ngay trong kênh adj*, đúng thứ mà injection test được dựng để bắt. Hai test `test_missing_stratum_contributes_neutral_pooled_share` và `test_large_prior_shrinks_to_pooled` pin lại; đừng nói chúng.

`apply_provenance` phải nhận `p_model` thật, không one-hot

`module_b_infer` chấm điểm lại 3 biến thể sentiment cho từng quad bằng teacher forcing để có `P_model(s|x)`. Với `--no-rescore`, fallback one-hot (`0.9/0.05/0.05`) khiến posterior **thành hàm tất định của (s dự đoán, φ)** và λ mất hoàn toàn ý nghĩa — số ra vẫn có vẻ hợp lý. Cờ đó chỉ để debug tốc độ; đừng để nó vào bất kỳ script sinh số chính thức, và đừng “tối ưu” bằng cách bỏ bước rescore.

Bootstrap: MỘT bản resample composition dùng chung cho mọi aspect

Vòng bootstrap trong `run` resample review trong strata một lần rồi tính slope cho **tất cả** aspect từ cùng bản resample đó. Đổi thành resample riêng từng aspect (trực giác “sạch hơn”) sẽ phá tương quan giữa các share của cùng một composition — các share cộng lại bằng 1, nên chúng **phải** chia sẻ cùng phương án resample. Cũng nhanh hơn `|aspects|` lần.

Reservoir sampling (Algorithm R) trong hai hàm lấy mẫu audit

`d0_sample` và `quad_audit_sample` dùng `seen[key] += 1 + j == rng.randrange(seen[key])` để mọi phần tử của ô có xác suất được giữ **đúng bằng nhau** trong một lượt đọc duy nhất. Thay bằng xác suất giữ cố định (`if rng.random() < 0.01`) — cách viết trực quan hơn — làm mẫu lệch về các dòng đến sau, tức lệch theo hotel/thời gian trong file store. Comment trong code đã ghi lý do; đừng “rút gọn” nó.

E3a: xáo timestamp làm nền null trước, rồi mới tiêm composition

Thứ tự trong `main` là `inject_shuffle` → `inject_composition`. Tiêm thẳng lên dữ liệu thật thì không thể đòi “adj im lặng”, vì dữ liệu thật *có thể có drift thật* — và verdict sẽ FAIL vì lý do sai. Nhánh `INCONCLUSIVE` (cả raw và adj đều im) cũng phải giữ: gộp nó vào PASS là tự cho điểm khi injection quá yếu. (Việc cần sửa là *cách shuffle* — #6 — không phải thứ tự này.)

Ghi chú khi test lại

- **Baseline hiện tại:** `PYTHONPATH=src python -m unittest discover tests` → **37 tests, OK, 0,08s**. Mọi thay đổi phải giữ 37/37 xanh trước khi chạy smoke.
- **Test cần thêm cùng lúc với fix** (hiện không có test nào bắt được 4 lỗi P1 sau): `linearize/parse` round-trip với term chứa `|` và `</quad>` (#8); parity `quad_uid` giữa `make_audit_samples` và `fit_bridge` (Phần C); parity thứ tự đặc trưng bridge `fit` ↔ `apply` (#4); `inject_shuffle` giữ mọi quad cùng `review_uid` ở cùng period (#6); `find_ckpt` chọn `round2` khi có cả round1/round2 (#9).
- **Bất biến số học phải giữ nguyên sau refactor:** `sum(clr(...).values()) == 0`; `adj == raw` khi chỉ có 1 stratum; `min_n` → ∞ ⇒ `adj` → share gộp; `v_raw` = tỷ lệ gộp của kỳ; tổng `gold` của `by_language` phải bằng `overall.gold` (hiện KHÔNG bằng — #17).
- **Trước khi chạy smoke test:** backup `outputs/drift/` hoặc set `PRISM_WORK_DIR` sang thư mục tạm — hiện smoke ghi đè file kết quả thật (#2). Đây là bước bắt buộc cho tới khi #2 được sửa.
- **Idempotency:** `--step all` bỏ qua step đã có output theo `sentinel()`. Kiểm lại rằng sentinel của `drift` (`drift_results.<cohort>.<level>.json`) đúng bằng đường dẫn `module_d_drift` thật ghi ra khi orchestrator **không** truyền `--out` — nếu sau này thêm `--out` vào nhánh drift mà quên sửa sentinel, `all` sẽ chạy lại drift vô hạn mỗi lần.
- **Kiểm chứng go/no-go end-to-end:** cố tình đặt `GO_NOGO["bridge_spearman_min"] = 0.99`, chạy `--stage bridge` rồi `--stage apply`, và xác nhận `w == conf_seq` (hiện tại sẽ KHÔNG — bridge vẫn được dùng, #3).

Còn treo — làm trước khi chạy số cuối

Thứ tự đề nghị P1 trước

Ngay: #1 (1 phút, chặn commit 1,15 GB) → #2 (bảo vệ file kết quả) → #9, #10 (chặn 6 tiếng GPU chạy sai/chết) → #3, #4 (số w của paper) → #5, #7 (định nghĩa thống kê được báo cáo) → #6 (negative control) → #8 (recall extractor). **Trước khi giao annotate:** #3 và parity `quad_uid` ở Phần C — sai khoá sau khi đã gán 300 quad là mất công người, không mất công máy. **Trước khi submit:** #11, #18, #19, #21 — đây là những chỗ reviewer đối chiếu code với mô tả và thấy lệch.

D-a (hiệu chỉnh recall) vẫn là stub P1

README §5 đã tự nhận. Cần: tập audit D0 gán xong → `recall_table.json` → **viết bước áp thật** (nhân $1/\hat{p}(\text{sentiment}, \text{length_bin})$ vào `w` ở `load_quads`, nơi đã có `n_words` trên từng quad) → chạy lại toàn bộ Module D. Cho tới lúc đó, mọi số v/π là **chưa hiệu chỉnh recall**, và log hiện tại nói ngược lại (#7). Không submit số cuối trước khi bước này xong hoặc được ghi rõ là giới hạn của bài.

Human audit 300 quad chưa có → nhánh Spearman chưa từng chạy P2

`fit_bridge` chỉ tính Spearman khi `audit_file` tồn tại và có ≥ 30 cặp khớp. Nghĩa là ngưỡng go/no-go thứ ba (`bridge_spearman_min = 0.30`) **chưa bao giờ được thực thi trên dữ liệu thật**, và đường code đó chưa được chạy lần nào. Chạy thử với 30 dòng audit giả (gán ngẫu nhiên) chỉ để xác nhận nhánh không crash và khoá join khớp — trước khi giao việc cho người.

Tên file `vimg` và `audit_sample_300 / human_audit_300` chưa thống nhất P2

`make_audit_samples` ghi `audit_sample_300.jsonl`; default `--audit` của module C là `human_audit_300.jsonl`; orchestrator Kaggle nối hai tên đó bằng một lần `copy_opt` có đổi tên. Chạy local mà quên truyền `--audit` ⇒ file không tồn tại ⇒ khối Spearman bị bỏ im lặng và bridge NO-GO vẫn được ghi (#3). Chọn một tên duy nhất, đặt hằng số trong `config.py`.

Baseline: 37/37 unit test PASS trên commit `7f5bc5d` (Python 3.14, không cần torch). Chưa chạy `smoke_test` (cần dữ liệu thật + sẽ ghi đè kết quả drift — xem #2). Kết luận trong doc dựa trên đọc code + kiểm chứng trực tiếp bằng script cho các mục #8, #9, #10, #13, #18, #19; các mục còn lại suy ra từ luồng dữ liệu. Quy ước: **Vị trí ghi theo tên hàm, không theo số dòng** — số dòng hết hạn ở commit sau, tên hàm thì không.